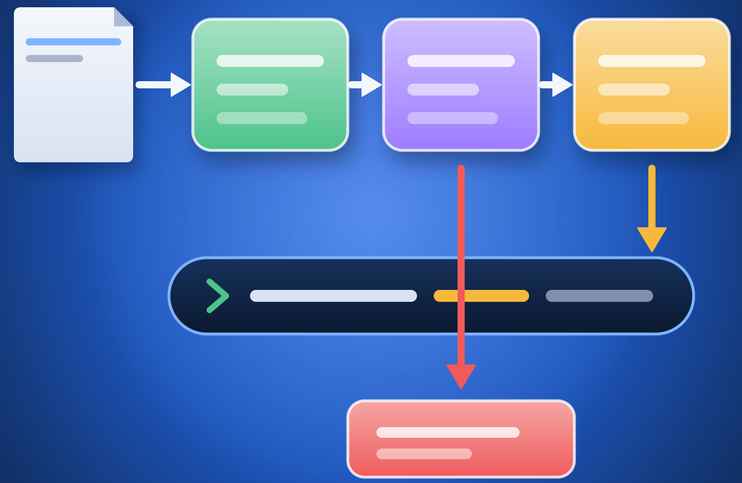
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 4

Redirection, Pipelines and Processes

Lab 4 guide: send output and errors to different places, connect commands, and build a report you can trust.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Use redirection and pipelines, and tell stdout, stderr and the exit status apart.
2. Filter and aggregate a CSV file, checking each stage of the pipeline.
3. Find out why a selection or a total is wrong, using normal and changed inputs.

THIS LAB

Lab 4

First the concepts, with examples. Then the lab on the server. Watching and signalling processes are optional extensions, so they come at the end.

DISCUSSION

A command prints an error and a result. Where does each go?

Both appear on your screen. Are they the same thing? How would you save only one of them?

CONCEPTS FIRST

Streams, pipes and filters

Three ideas: every program has three streams, a pipe joins two programs, and small filters do one job each.

Every command has three streams

Stream	Number	Default	Used for
stdin	0	The keyboard	Input
stdout	1	The terminal	Normal output
stderr	2	The terminal	Error messages and diagnostics

Both output streams appear on the screen, so they look the same. They are different streams, and you can send them to different places.

The redirection operators

Operator	Meaning	Example
`>`	Send stdout to a file, replacing it	`ls > files.txt`
`>>`	Append stdout to a file	`echo done >> log.txt`
`2>`	Send stderr to a file	`ls /fake 2> errors.txt`
`2>&1`	Make stderr go where stdout goes now	`cmd > all.txt 2>&1`
`&>`	Bash shorthand: stdout and stderr into one file	`cmd &> all.txt`
`<`	Read stdin from a file	`sort < unsorted.txt`
`/dev/null`	Throw the output away	`cmd > /dev/null 2>&1`

A single > replaces the file. Use >> to add to it. In a pipeline, |& sends both streams to the next command.

Two streams, two files

student@server: ~/oslab-work/lab4

```
$ printf 'normal output\n' > evidence/stdout-demo.txt
$ cat missing-demo.txt 2> evidence/stderr-demo.txt
$ cat evidence/stdout-demo.txt
normal output
$ cat evidence/stderr-demo.txt
cat: missing-demo.txt: No such file or directory
```

printf writes the text. > sends it into the first file. The second command fails on purpose, and 2> catches only its error message.

Nothing about the error appeared on the screen the first time. It went into the file.

The order of redirections matters

stderr follows stdout to the file

student@server: ~/oslab-work/lab4

```
$ bash -c 'echo normal; echo problem >&2' > both.txt 2>&1
$ cat both.txt
normal
problem
```

First stdout goes to the file. Then 2>&1 points stderr at that same place.

stderr stays on the screen

student@server: ~/oslab-work/lab4

```
$ bash -c 'echo normal; echo problem >&2' 2>&1 > only-out.txt
problem
$ cat only-out.txt
normal
```

Here stderr is pointed at stdout while stdout is still the screen. Only then does stdout move to the file.

Read redirections from left to right.

A pipe joins two commands

- The pipe | connects the stdout of one command to the stdin of the next.
- The commands run at the same time. Data flows through as it is produced.
- Only stdout goes through the pipe. stderr still goes to the terminal.
- The exit status of a pipeline is, by default, the status of the last command.



Read it stage by stage: all rows, then only the ok rows, then only field 2 of those rows.

Small filters, one job each

Command	What it does	Example
grep	Keeps the lines that match a pattern	`grep '^ok,' events.csv`
cut	Takes fields from each line	`cut -d, -f2`
awk	Fields, conditions and arithmetic	`awk '{ total += $1 } END { print total+0 }'`
sort, uniq -c	Sort lines, then count equal neighbours	`sort data | uniq -c`
wc -l	Counts lines	`wc -l file.txt`
head, tail	The first or last lines	`head -n 1`
tee	Saves a copy and passes the data on	`ls | tee files.txt | wc -l`

uniq compares neighbours only, so sort first. tee lets you look inside a pipeline.

grep, cut and awk

grep: keep matching lines

- grep -v: lines that do not match.
- grep -c: only the count of matching lines.
- grep -i: ignore upper and lower case.
- grep -E: extended patterns.

cut: simple fields

- -d sets the delimiter, -f picks the fields.
- cut -d: -f1 /etc/passwd lists account names.
- It cannot do arithmetic or conditions.

awk: fields with logic

- $1 is the first field, $2 the second.
- END runs once, after the last line.
- awk -F: '$3 >= 1000' /etc/passwd keeps ordinary accounts.

Use cut for simple delimited fields. Use awk when you need a condition or a sum.

Rows are not counts

student@server: ~/oslab-work/lab4

```
$ printf 'ok,3\nfail,2\nok,5\n' | grep '^ok,' | wc -l
2
$ printf 'ok,3\nfail,2\nok,5\n' | grep '^ok,' | cut -d, -f2 | awk '{ t += $1 } END { print t+0 }'
8
```

wc -l counts the selected rows: two lines start with ok.

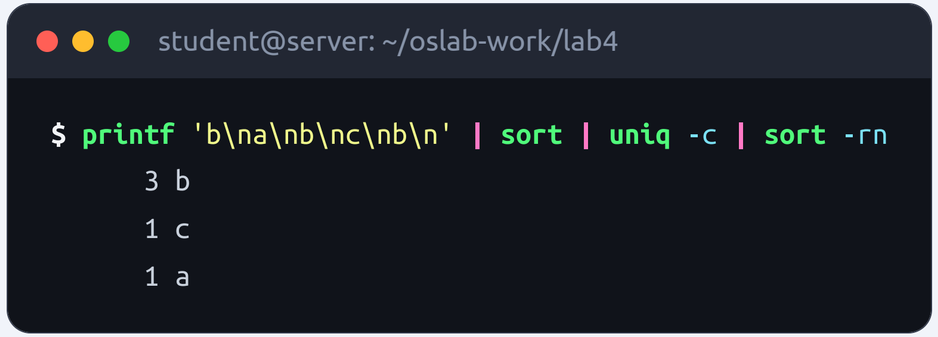
The report needs the sum of the second field of those rows: 3 plus 5.

REMEMBER

Know which quantity you want

Counting rows and adding numbers answer different questions. Check each stage before you trust the last one.

The frequency pipeline



Extract a column, then sort | uniq -c | sort -rn. It counts how often each value occurs and puts the most frequent first.

The first sort puts equal lines next to each other, uniq -c counts them, and sort -rn orders the counts from high to low.

LAB 4

A report you can trust

TechCorp receives small CSV event logs. Count the successes, and keep results apart from diagnostics.

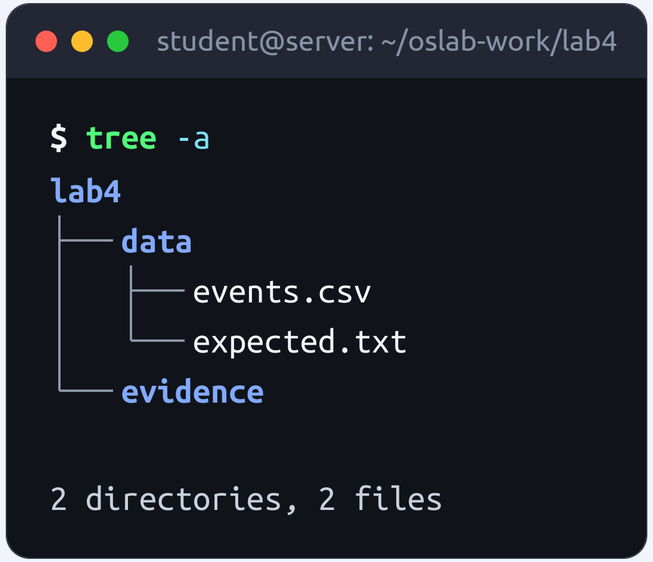
The scenario

SCENARIO

Event logs at TechCorp

Produce a trustworthy success-count report from a small CSV event log, and distinguish command output from diagnostics.

The core uses text files only. Process-management examples are optional and use your own processes.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 18	Task 1, guided: redirection
18 to 25	Task 2, guided: pipelines and filters
25 to 35	Written prediction

Minutes	What happens
35 to 70	Task 3: build the report
70 to 85	Normal and edge tests, feedback
85 to 100	Individual changed-case checkpoint
100 to 120	Correction, cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Privileged and boot work is never done on the shared server.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

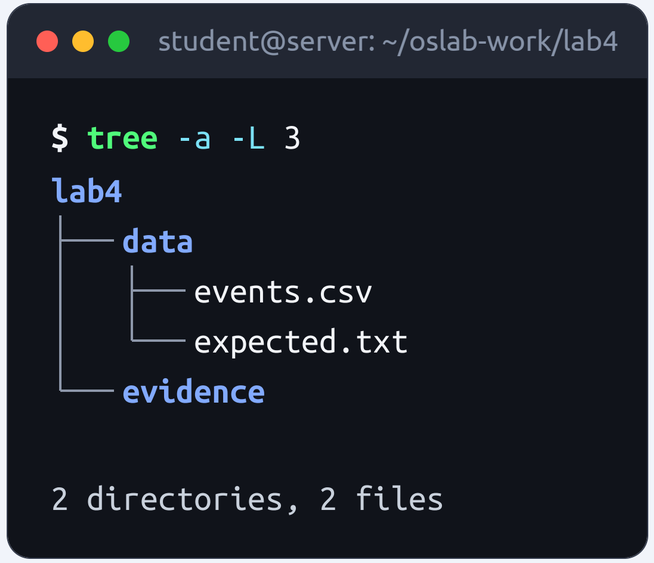
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab4
$ cd "$OSLAB_WORKSPACE/lab4"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab4 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



data/events.csv has a header and three records.

data/expected.txt holds the report the lab asks for: ok total: 8.

The helper also writes a hidden file, .oslab-managed.json. Leave it as it is.

GUIDED

Redirection, then pipelines

Tasks 1 and 2, with the instructor: ten to twenty-five minutes.

Task 1: normal output and an error, apart

student@server: ~/oslab-work/lab4

```
$ printf 'normal output\n' > evidence/stdout-demo.txt
$ cat missing-demo.txt 2> evidence/stderr-demo.txt
$ cat evidence/stdout-demo.txt
$ cat evidence/stderr-demo.txt
```

The missing-file command fails on purpose. > redirects descriptor 1 (stdout), 2> redirects descriptor 2 (stderr), and >> appends.

TRY

Observe

Which file holds the error text? What did the terminal show while the second command ran?

Task 1: both streams into one file

student@server: ~/oslab-work/lab4

```
$ bash -c 'printf "out\n"; printf "err\n" >&2' > evidence/both.txt 2>&1
$ cat evidence/both.txt
```

bash -c runs the quoted text as a small program. The second printf writes to stderr: >&2.

2>&1 makes stderr refer to stdout's current destination, the file. Reversing the two redirections can change where stderr goes.

Task 2: a pipeline, one stage at a time

student@server: ~/oslab-work/lab4

```
$ printf 'ok,3\nfail,2\n'
$ printf 'ok,3\nfail,2\n' | grep '^ok,'
$ printf 'ok,3\nfail,2\n' | grep '^ok,' | cut -d, -f2
```

Add one stage at a time, and look at the output before you add the next. The last stage should print 3.

| forwards stdout to the next command. stderr is not included by default.

Task 2: look at the real input

The lab uses simple two-field CSV without quoted commas.

student@server: ~/oslab-work/lab4

```
$ cat data/events.csv
$ cat data/expected.txt
```

TRY

Observe

Does wc -l count successful events or selected rows? Which stage would add numbers instead of counting records?

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

What will grep '^ok,' data/events.csv | cut -d, -f2 | head -n 1 print? Will that equal the required successful total? Explain what information it discards.

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run the command. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

TASK 3

Build the report

Thirty-five minutes, from minute 35 to 70. Work alone.

Step 1: run the incomplete pipeline you predicted

student@server: ~/oslab-work/lab4

```
$ grep '^ok,' data/events.csv
$ grep '^ok,' data/events.csv | cut -d, -f2
$ grep '^ok,' data/events.csv | cut -d, -f2 | head -n 1
```

Compare each stage with the original data.

TRY

Observe

Which stage first loses information? What does head -n 1 keep, and what does it throw away?

Step 2: write your own report.sh

What report.sh must do

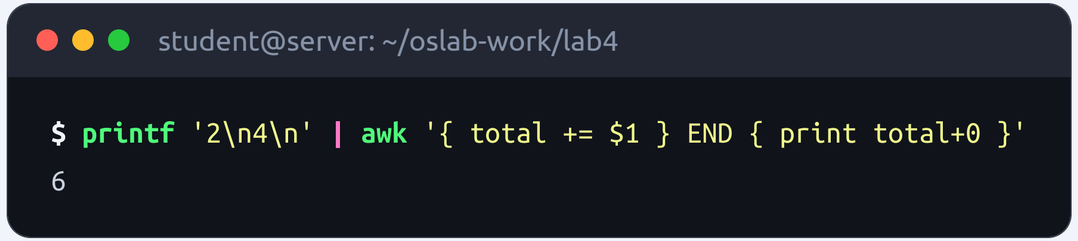
- Take one CSV path as its argument.
- Select every record whose first field is exactly ok.
- Sum field 2 of those records.
- Print ok total: NUMBER once.
- Reject a missing input, and print zero when no row matches.

YOUR TURN

You write it

Nobody gives you the script. Decide where selection, field extraction and summing belong. A staged pipeline or one carefully limited awk program both work.

Step 3: awk as a calculator



total += $1 adds the first field of each line. END runs once, after the last line. total+0 prints 0, not an empty line, when nothing arrived.

This demonstration teaches summing without giving you the whole solution.

Step 4: run it and compare

student@server: ~/oslab-work/lab4

```
$ bash report.sh data/events.csv > evidence/normal.txt
$ diff -u data/expected.txt evidence/normal.txt
```

diff prints nothing when the two files are identical.

TRY

Think

A matching report is useful evidence. Why does matching one file not prove that the script is right for every CSV?

Hints, one at a time

DONE

Complete when

report.sh prints ok total: 8 for the supplied file, and you can say what each stage of it does.

If you are stuck

1. Tell rows apart from counts.
2. Print each stage of the pipeline.
3. Sum only the selected numeric field, and format the label at the end.

AI is allowed here. Test what it suggests on these small files.

Change the input, run it again

Fifteen minutes, from minute 70 to 85.

Make two changed inputs

student@server: ~/oslab-work/lab4

```
$ printf 'status,count\nfail,2\n' > data/no-ok.csv
$ cp -- data/events.csv data/changed.csv
$ printf 'ok,7\n' >> data/changed.csv
```

no-ok.csv has no ok row. changed.csv is a copy of the original with one more record, ok,7.

The original fixture is not altered. >> adds a line at the end.

Run the report on every input

student@server: ~/oslab-work/lab4

```
$ {
$   bash report.sh data/no-ok.csv
$   bash report.sh data/changed.csv
$   bash report.sh data/missing.csv
$   printf 'missing-input status=%s\n' "$?"
$ } > evidence/edge.txt 2>&1
$ cat evidence/edge.txt
```

Expect ok total: 0 and ok total: 15. The missing file must fail.

$? is the exit status of the last command. 2>&1 puts the error message in the file too.

What each test contradicts

Case	Expected	A wrong script that it exposes
Normal	ok total: 8	Counting rows instead of adding numbers
No ok row	ok total: 0	Printing nothing, or an error, when nothing matches
Changed count	ok total: 15	Keeping only the first match
Missing input	An error and a non-zero status	Ignoring input errors

The exercise does not cover quoted fields in CSV. State that limit in your explanation. grep returns 1 when nothing matches, which is not the same as a read error.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

The original fixture gains ok,7 and fail,20. Predict the report total. A student uses wc -l after filtering. Identify why their answer would measure a different quantity.

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

The instructor collects your answer before discussing the public key.

CORRECTION AND SUBMISSION

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) Why is the row count different from the summed event count? (b) Which stream carries a missing-file error? (c) Which inputs are outside your parser assumptions?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab4/evidence"
$ cd "$OSLAB_WORKSPACE/lab4"
$ cp -- report.sh "$SUBMISSION_REPO/lab4/"
$ cp -- evidence/normal.txt evidence/edge.txt "$SUBMISSION_REPO/lab4/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the script, the two evidence files and two selected test records, not every terminal output. The core starts no background job.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab4
├── evidence
│   ├── edge.txt
│   └── normal.txt
├── README.md
└── report.sh

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Correct report selection and aggregation, and clear stdout and stderr handling	3
No-match, changed-count and missing-input diagnosis	2
Each pipeline stage explained, and the claim each test supports; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
wc -l after the filter	The number of rows, not the total	Add field 2 with awk
Only the first match kept	A total that is too small	Do not use head -n 1 to sum
Error text mixed into the report	A message inside ok total lines	Keep stderr apart from stdout
Trusting the pipeline's success	A zero status while a stage failed	Inspect the intermediate values

Processes, jobs and signals

Optional extensions. They use only your own, bounded processes.

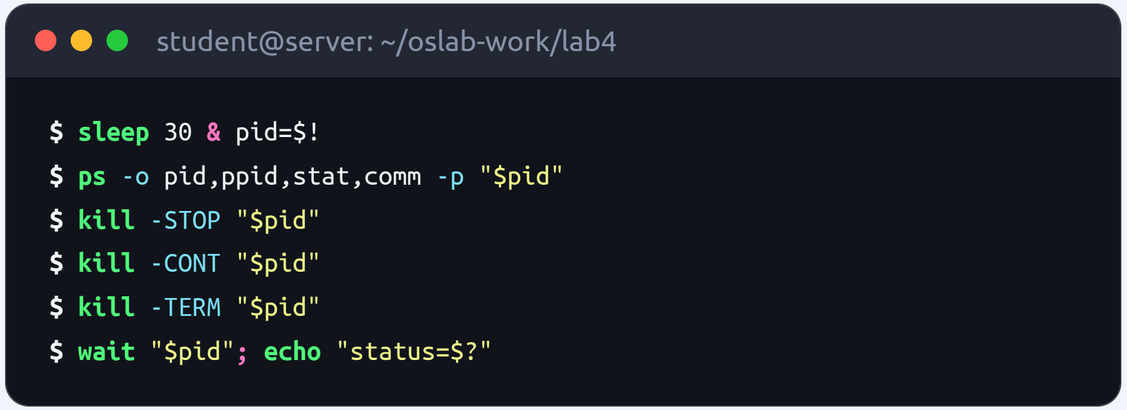
Looking at processes

Command	What it shows	Output
`ps aux`	A snapshot of all processes	Can be redirected
`ps --forest`	The process tree	Can be redirected
`ps -o pid,ppid,stat,comm -p PID`	One process, with its parent and state	Can be redirected
`top, htop`	A live view, updated all the time	Interactive: take a screenshot

Start exactly one process of your own and keep its PID. Do not kill processes by name.

Jobs and signals

Signal	Number	Effect	Can be caught?
SIGTERM	15	Asks the process to end	Yes
SIGKILL	9	Ends it at once	No
SIGSTOP	19	Pauses it	No
SIGCONT	18	Resumes it	Yes
SIGINT	2	Ctrl+C from the keyboard	Yes



$! is the PID of the last background job. A stopped process shows state T. A wait status above 128 means a signal ended it.

Job control

student@server: ~/oslab-work/lab4

```
$ sleep 60 &
$ jobs -l
$ fg %1
$ # press Ctrl+Z to pause it
$ bg %1
$ nohup sleep 60 &
```

A trailing & runs a command in the background. jobs -l lists your jobs with their PIDs.

fg %1 brings job 1 to the front. Ctrl+Z pauses the front job, and bg %1 lets it run in the background.

nohup keeps a command running after the terminal closes. Use these only on your own, short commands.

Orphans and zombies

	Orphan	Zombie
Who ends first?	The parent	The child
Is the child still running?	Yes	No, it has exited
State in ps	S or R, normal	Z, shown as defunct
Who cleans it up?	The system gives it a new parent, often PID 1	Its parent must call wait
Can you kill it?	Yes, it is a normal process	No, it is already dead: fix the parent

Run these demonstrations for a few seconds, in a disposable environment, and leave nothing running.

Key takeaways

01

Three streams

stdin, stdout and stderr are separate. Redirect each on purpose.

02

Pipes

A pipe carries only stdout. Build a pipeline one stage at a time.

03

Rows are not counts

Check which quantity each stage produces before you trust the total.

04

Test the edges

No match, a changed count and a missing input show what the script really does.

Next session

COMING UP

Lab 5: threads, kernel workers and signals

Look at threads inside a process, the kernel's own workers, and how a signal reaches a process.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 3, Processes.